

HiGPose: A Hierarchical Geometric Feature Extractor for 6D Object Pose Estimation

Bono Giorgio
s343572

Camolese Claudio
s344788

Chen Enrico
s337750

Politecnico di Torino Politecnico di Torino Politecnico di Torino

s343572@studenti.polito.it s344788@studenti.polito.it s337750@studenti.polito.it

Abstract

This project addresses the task of 6D object pose estimation on the LINEMOD preprocessed dataset. In the initial phase, a baseline architecture is implemented by combining YOLOv11n object detector, a pretrained ResNet-based network to extract colour features and a final MLP to predict rotations and translations.

As an extension, the approach is enriched by incorporating depth information. HiGPose, that is a DenseFusion-inspired architecture, is implemented to process RGB-D images to retrieve pixel-wise dense feature embeddings, which are used for estimating the pose. Feature embedding is composed by color and geometric features. The first ones are extracted from the cropped images by ResNet backbone while the second ones are computed from the point clouds by a hierarchical multi-stage network, based on PointNet++.

HiGPose processes point clouds iteratively by following grouping, neighborhood aggregation and geometric features extraction for three different point cloud density levels.

The resulting multi-scale features are up-sampled and aligned to the original resolution.

Moreover, a potential application in autonomous driving of the proposed model is explored. In this practical scenario, 6D poses of cones need to be predicted by using a LiDAR and two cameras.

1. Introduction

6D Pose Estimation is crucial task in many Computer Vision applications, such as autonomous driving, virtual reality, and robot arm grasping [8, 14], consisting in estimating an object’s position and orientation (6 Degrees of Freedom). In autonomous driving the purpose is to enhance the perception of a vehicle’s surroundings, by detecting nearby obstacles and assist in safe driving. In the virtual reality case, the goal is to provide to users a smoother experience.

Instead, in robot arm grasping, the main objective is to recognize an object, localize, and grasp it, and this application can be utilized in fields ranging from manufacturing industry, logistics, to healthcare.

In other words 6D position estimation plays a fundamental role in this domain. Depending on each specific use case, additional constraints and requirements may be imposed such as real time or near real time performances, sub-millimeter accuracy, and stable performance in different lightning conditions.

Unfortunately, the task itself is not simple as many issues can affect accuracy, robustness, and the ability of a model to generalize to unseen objects. Primary factors are, among others, occlusions, auto-occlusions, object’s sizes, lighting conditions, symmetry, and cluttered scenes [8, 14].

Over the years, numerous studies have been published. While certain approaches address particular issues, others focus on different aspects; consequently, developing a comprehensive solution that excels in all dimensions still remains difficult. To evaluate the models, a range of standard datasets are available as benchmarks, such as LINEMOD, KITTI, and YCB Object.

In this work, an object detector model based on YOLOv11 [5, 7, 11] is first combined with a ResNet [2] backbone and some FC layers to provide baseline results using RGB-only images, subsequently the approach is extended to incorporate depth information by hierarchically extracting geometry features with PointNet++ [10] and fusing them with colour features with a technique inspired from DenseFusion [12], resulting in an improved performance.

Although inspired from DenseFusion [12] for its robust behavior in case of occlusion and cluttered scene [14], and near real-time inference [12], this work differs from it under several aspects, among them are: instead of semantic segmentation an object detector is used, iterative refinement module is not implemented, and PointNet++ is used instead of PointNet [9] for better local features extraction.

After this introduction, some related works are presented

in Sec. 2, following there are descriptions of the architecture (Sec. 3) of both baseline (Sec. 3.1.1) and extension (Sec. 3.2) models, experimental results and training procedures (Sec. 4), a possible application to autonomous driving (Sec. 5) and, finally, conclusion and possible future works (Sec. 6).

The code can be found at this link (https://github.com/BonoGiorgio02/6DPose_Estimation).

2. Related Works

Over the years, extensive research has been done on *6D Pose Estimation* due to its importance in *Computer Vision*. The pose estimation methods can be mainly classified into traditional methods and learning methods.

Traditional methods include *Template-based* and *Feature-based* methods. The former consists in storing in a database a variety of synthetic renderings of an object in different positions and orientations. During inference the object image is matched with each of its available renderings, the closest one is chosen and the pose established. Although this method may perform well with texture-less objects, it is highly sensitive to lighting variations and occlusions, and the accuracy of predictions depends on the exhaustiveness of the database [8]. A work of this category is *LINEMOD* [3] from which the used dataset is named after.

The latter one focuses on the local features called *keypoints*, which are extracted from the images by using algorithms like *SIFT*. These points are matched with *3D* keypoints from a *3D* model, and eventually these two set of points are given to algorithms like *PnP* to predict the pose. These methods works also in cluttered scenes, but their accuracy depends on the quality of extracted keypoints, and the *6D* poses are often coarse, thus requiring an additional iterative procedure to refine the predictions and making these methods not suitable for real-time scenarios [8].

With *Deep Learning* being more and more popular, researchers have introduced also *learning-based* methods that typically achieve higher accuracy, but at the same time requiring larger datasets and computationally demanding. These methods can be *classification-based* where the pose space is discretised [6] or *regression-based* where pose is directly regressed with a *CNN*. *PoseCNN* [13] belongs to the latter category, it can receive as input both *RGB* and *RGB-D* images, but to achieve higher accuracy depth information must be available, such that *Iterative Closest Point* (ICP) procedure can be applied to refine the pose, making this method very slow. Another approach is *DenseFusion* [12] that receives two input sources, processes them separately and fuse them to extract features. It outperforms *PoseCNN+ICP*, it is much faster and robust to cluttered scenes. A work inspired from *DenseFusion* is [1], it additionally adds a mask to enhance the semantic informa-

tion and an attention module, and although it outperforms *DenseFusion*, due to larger number of parameters it needs a longer training time.

3. Model

The objective of this work is to predict *6D* of objects given *RGB(-D)* images. Two approaches are presented: a *baseline* approach where *RGB-only* images are given as input and an *extension* of it by incorporating depth information. In both cases the *RGB-only* images are cropped by an object detection model which is trained independently from these two pose estimation modules. In the extension approach, the depth information is used, more specifically a segmentation mask is used to extract pointclouds.

The output is represented by a translation vector and a rotation vector in quaternion format (similarly in [13]), as quaternions offer multiple advantages over rotation matrices, such as: improved computational efficiency due to the more compact representation format and reduced number of parameters, and easier orthonormality constraint.

The object detection module used is *YOLOv11* [11], which is a pretrained model. More specifically, the *nano* version is used as it is the most lightweight model available.

3.1. Baseline Pose Module

The baseline approach uses just *RGB* images. The architecture is described in Sec. 3.1.1.

3.1.1 Architecture

For the baseline architecture different variants are tested. These variants includes the use of: a pretrained *ResNet18* backbone without freezing and two single layer MLP respectively for rotation and traslation; a pretrained *ResNet18* backbone without freezing its weights, with two deeper MLP. For each one of them, both loss functions are used. The architecture receives as input the cropped image and extracts color features thanks to *ResNet18* backbone. The features are given to the two MLP heads in order to make prediction of rotation and translation.

The various possible combinations of the baseline architecture, combined with different loss functions, are tested over 20 epochs. After this initial phase, the architecture that produces the best results is trained for additional epochs.

3.1.2 Loss Function

The first loss function used is computed as weighted sum of the translation loss and rotation loss. Each of these two losses is the *Mean Squared Error* (MSE), so the equation is

$$\mathcal{L}_{\text{MSE}} = \frac{1}{3} \sum_{i=1}^3 (\hat{t}_i - t_i)^2 + \frac{1}{4} \sum_{i=1}^4 (\hat{q}_i - q_i)^2$$

where $(\hat{t}_i)$ is predicted translation vector, (t_i) ground truth translation, $(\hat{q}_i)$ predicted rotation and (q_i) ground truth rotation in quaternion representation.

In an effort to improve the performance, the use of another loss function is explored, more specifically it is computed as Mean Squared Error (MSE) between the predicted pose and the ground truth pose, as shown in the following equation:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_i [(R \cdot x_j + t) - (\hat{R}_i \cdot x_j + \hat{t}_i)]^2$$

where $N = 1000$ is the number of sampled points of the CAD model of the object, R and t are the ground truth rotation and translation matrix, while, $\hat{R}$ and $\hat{t}$ are their corresponding predictions.

3.2. Extension Pose Module

As an extension also the depth information is used to predict the pose. The architecture is described in Sec. 3.2.1, and loss function in Sec. 3.2.2.

3.2.1 Architecture

The *HiG*'s architecture is inspired from [12], but no refinement module is implemented and *PointNet++* is used. The complete pipeline of the pose predictor model is depicted in Fig. 1, while the architecture of *PointNet++* is depicted in Fig. 2.

The module receives as inputs cropped image and masked depth.

The cropped image is given to a backbone based on *ResNet18* (not considering the last *FC* layer and average pooling to maintain spatial information). To enhance the model's focus on geometric embeddings, the extracted color features are down-sampled from 512 to 256 features. Subsequently, each cropped image is resized to be of size $H_{\text{pad}} \times W_{\text{pad}}$. From the set of valid points obtained from the masked depth map, specifically those that lie within both cropped and padded image and possess valid depth value, only 600 are randomly selected.

Regarding geometric embeddings, among all points of the pointcloud, only 800 of these points are randomly chosen to make training less computationally expensive. These points are given to the *PointNet++* module consisting of 3 *set abstraction* layers to hierarchically extract local features and progressively produce higher level features.

Each *set abstraction* layer consists of 3 distinct phases:

1. *sampling* phase: using *Farthest-Point-Sampling* (FPS) the *centroids* are computed (across these 3 *set abstraction* layers the number of centroids to be computed are 800, 200, 50). *Random sampling* is an alternative method to compute centroids, but *FPS* allows better coverage of all points [10]

2. *grouping* phase: points are grouped into neighborhoods according to the computed centroids
3. *PointNet layer*: local features are extracted by retaining only the centroids whose features are the largest ones in relative sense within the neighborhood. This can be represented mathematically by:

$$\text{centroid}_i = \max_{j \in \mathcal{N}} \text{MLP}([h_j, \text{pos}_j - \text{pos}_i])$$

where $\mathcal{N}$ is set of neighbors of centroid centroid_i , h_j is the value of the features extracted in the previous layer of j^{th} point, pos_i is the positional value $[xyz]$ of centroid i while pos_j the one of j^{th} point in the neighborhood of centroid i and *MLP* used is

$$[FC - \text{BatchNorm} - \text{ReLU}] \times 2$$

The features extracted at these 3 different levels are up-sampled to obtain the following shapes $[B \cdot 800, 64]$, $[B \cdot 800, 128]$, $[B \cdot 800, 256]$, then they are concatenated. Finally, by considering the same 600 points previously selected, color embeddings and geometric embeddings are fused pixel-wise.

These features are given to 3 different heads (*pose predictor*) to predict translation, rotation and confidence score. Each head consists of the following layers:

$$[\text{Conv} - \text{BatchNorm} - \text{ReLU}] \times 3 - \text{Conv}$$

each head differs only by the size of the output of the last convolutional layer. The pose with the highest confidence is chosen to be the global predicted pose for the entire object.

3.2.2 Loss Function

The loss function used is the one that can be found in [12] and it can be defined as:

$$L = \frac{1}{N} \sum_i (L_i^p \cdot c_i - w \cdot \log(c_i))$$

where N is set of dense pixels, c_i confidence of dense pixel and $w \cdot \log(c_i)$ a regularization term. L_i^p is defined for each dense pixel as

$$L_i^p = \frac{1}{M} \sum_j \|(R \cdot x_j + t) - (\hat{R}_i \cdot x_j + \hat{t}_i)\|$$

where $[R, t]$ is the ground truth pose and $[\hat{R}_i, \hat{t}_i]$ predicted pose of the i^{th} dense pixel, x_j is one of the $M = 1000$ points selected with *FPS* from the 3D model of the detected object. Not all points are selected due to high computational cost, and it can be observed that L_i^p is the *ADD* metric.

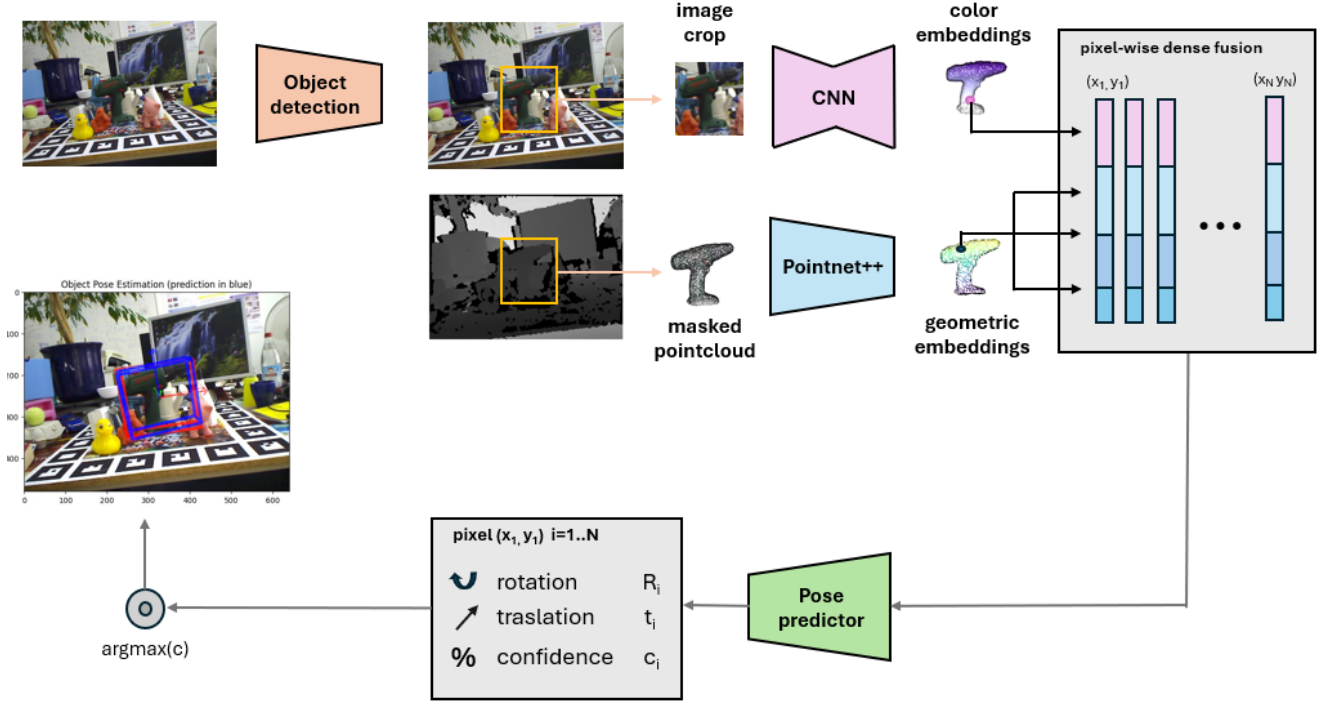


Figure 1. Complete pipeline of the extension architecture, considering the input image, the object detection module, the pose prediction module, and the final image with the 3D bounding box of both ground truth (red) and predicted pose (blue)

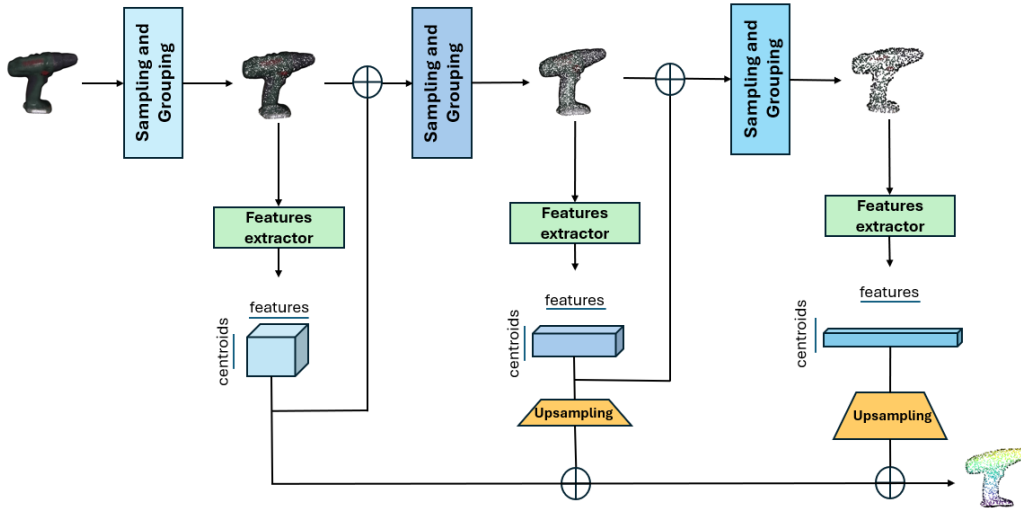


Figure 2. Architecture of *PointNet++*, considering the input pointcloud, the *set abstraction layers* with sampling and grouping, and feature extractor. For higher level features there is also upsampling step

4. Experimental Results

In this section the experimental setup is presented. Firstly, the used dataset is described and extensively explored. Secondly, all the necessary preprocessing steps are described. Also the metric used for evaluation is de-

scribed, followed by a detailed discussion of both baseline and extension pose prediction modules, including their experimental setup and results.

4.1. Dataset Exploration

The dataset used is the *preprocessed LINEMOD dataset*. Among the 15 distinct objects contained in the original *LINEMOD* dataset [4], only 13 are maintained. For each object an *RGB-only* image is available, together with its corresponding *depth*, *mask*, camera intrinsics, and ground truth information (translation, rotation matrix, bounding box in top-left *XYWH* format, object id). Additionally, each distinct object has a corresponding 3D model along with information like diameter, and size. The dataset is balanced, that is distribution of object type is nearly uniform, this avoids model to be biased towards more frequent classes.

The exploratory analysis reveals that some objects do not have both *RGB* and *mask*, so these are ignored in the extension phase.

Furthermore, although each object has its own camera information, this is redundant as they are identical.

To train, validate, and test the model (object detection, baseline and extension modules), the dataset is partitioned in a stratified manner into 3 sets: training (70%), validation (15%), and test set (15%).

4.2. Preprocessing

The exploratory analysis also reveals that each image belonging to class 02 contains multiple objects, and this poses some challenges as object occlusion and instances extending beyond the image boundaries (640×480 pixels). To mitigate these issues, only the object of class 02 is retained in each corresponding image, while the others are excluded.

To reduce computational overhead during training and evaluation, the ground truth normalized quaternions are computed and stored in the ground truth files.

In order to train *YOLOv11*, the required directory structure is created, bounding boxes normalized and stored in center-based *XYWH* format, with one annotation file per image. Additionally, a YAML file is also generated.

To enhance the stability of model training, input images are standardized by subtracting per-channel mean and dividing by per-channel standard deviation. Only training partition is considered to compute these values to avoid *Data Leakage*.

YOLOv11 is finetuned on the *LINEMOD* dataset for 50 epochs with *batch_size* = 64 and *img_size* = 640. Values of the non default hyperparameters are listed in Tab. 1. Evaluating on the test set, it yields *mAP50* = 0.995 for any classes and *mAP50* - 95 $\in$ [0.936, 0.984].

4.3. Metric

To evaluate the performance of the pose estimation models, the *Average Distance of Models Points* (*ADD* [13]) is employed. This metric evaluates the discrepancy between a predicted pose and its corresponding ground truth by computing the mean *Euclidean Distance* between 3D points

of the object model transformed using the predicted and ground truth poses. For non symmetrical objects the equation is:

$$ADD = \frac{1}{m} \sum_{j=1}^m \left\| (\hat{R} \cdot x_j + \hat{t}) - (R \cdot x_j + t) \right\|$$

where m is the number of 3D model points, $\{\hat{R}, \hat{t}\}$ is predicted pose, and $\{R, t\}$ ground truth pose. The rotation is represented as an orthonormal rotation matrix.

If the object is symmetric the *ADD-S* [13] is used, it compares each ground truth point to the closest predicted point to account for pose ambiguity introduced by different views. The equation is:

$$ADD_S = \frac{1}{|\mathcal{M}|} \sum_{x_2 \in \mathcal{M}} \min_{x_1 \in \mathcal{M}} \|(\hat{R} \cdot x_1 + \hat{t}) - (R \cdot x_2 + t)\|$$

where $\mathcal{M}$ is the set of model points.

If a predicted pose is considered correct when:

$$ADD < diameter \cdot threshold$$

or

$$ADD_S < diameter \cdot threshold$$

with *diameter* the diameter of the object and *threshold* = 0.1, then *accuracy* [14] can be defined as the following:

$$accuracy = \frac{number\ correct}{total\ predictions} \cdot 100\%$$

4.4. Baseline setup

The baseline module is trained for 52 epochs with *batch_size* = 32. The optimizer used is Adam with $lr = 1.0 \cdot 10^{-4}$ and *weight_decay* = $1.0 \cdot 10^{-5}$ and scheduler is *CosineAnnealingLR* with $T_{max} = 52$, $\eta_{min} = 1.0 \cdot 10^{-6}$.

Cropped images are padded by replicating the border pixel values such that they have same sizes in the entire batch.

4.5. Extension setup

The extension module is trained for 90 epochs with *batch_size* = 16. The optimizer used is AdamW with $lr = 1.0 \cdot 10^{-4}$ and *weight_decay* = $1.0 \cdot 10^{-5}$ and scheduler is *CosineAnnealingLR* with $T_{max} = 100$, $\eta_{min} = 1.0 \cdot 10^{-6}$.

Cropped images are padded with the same methodology described in Sec. 4.4.

4.6. Results

Results are listed in Tab. 2. Both baseline and extension models are trained on ground truth bounding boxes rather than the ones predicted by YOLO object detector, so the first two columns of our results in Tab. 2 should be considered as upper bounds.

Args	HSV_H	HSV_S	HSV_V	Flipud	Fliplr	Mosaic	Translate	Scale	Erasing	Patience	Dropout
Value	0.1	0.1	0.1	0.0	0.0	0.0	0.0	0.0	0.0	5	0.3

Table 1. *YOLOv11* non default hyperparameters

The table Tab. 2 demonstrates that the lack of geometric features leads to suboptimal results. More specifically the results refer to the best performing baseline model, that is the one composed by the pretrained *ResNet18* backbone without freezing its weights, deeper MLP and having weighted sum of translation and rotation MSE loss.

With the addition of geometric features, results are comparable to the state-of-the-art methods. By considering the entire pipeline, the results outperform the ones from *DenseFusion* without iterative refinement [12] for some classes (driller, eggbox, glue, lamp). It can be observed that *HiG* network works particularly well with symmetric objects (including cones for autonomous driving as in Sec. 5), while it might struggle with small objects. To be more precise, the network performs better on objects that are both symmetric, with non-uniform geometrical shapes, and with large size.

For some objects, the entire pipeline of the extension with the object detection module outperforms the corresponding upper bound module. This might be due to small errors on detected boxes, that can allow pose prediction module to focus on central area of each object.

5. Application to Autonomous Driving

The 6D pose estimation task is a widely used concept in the field of autonomous driving. Since two members of this project are part of the Squadra Corse Driverless team at Politecnico di Torino, a possible application of this architecture is explored, more specifically it is explored the possibility to integrate the architecture into the team’s simulator. This would allow the autonomous vehicle to predict the pose estimation of traffic cones used to delimit the track.

To get relevant information, specific ROS nodes of the car’s pipeline are listened. These nodes output images from the two onboard cameras and a point cloud which is filtered to highlight the cone profiles (obtained using a LiDAR sensor).

Data from several full laps of the track are recorded. The model’s architecture described in Sec. 3.2.1 is extended to include camera extrinsics information. This matrix is essential to properly fuse the information from images and the LiDAR point cloud since the two cameras are placed to cover a combined field of view of 120° while LiDAR is oriented straight ahead.

In order to achieve the objective, several challenges have been addressed. Firstly, collecting the data needed to train the model has required knowing how to use the ROS

robotics framework and being familiar with the Gazebo simulation environment.

Once the data has been collected, a data cleaning phase is performed. This is necessary due to calibration issues between the LiDAR and the cameras in the simulator, which may result in missing or inconsistent data.

For the object detection part, a YOLOv5 model pretrained on cones for formula students race is used. LiDAR sensor’s output is used as ground truth for translation of the cones.

Fig. 3 shows the same frame captured by the vehicle’s stereo cameras.

However, since the simulator does not allow obtaining ground truth for the rotation of the cones, and considering that the objects are also symmetric, the identity matrix is used as the ground truth for the rotation. Inside the car’s team, an ICP algorithm is being developed. This algorithm, together with the CAD model of the object, allows obtaining ground truth for the cone in order to train on the rotation as well. However, so far, this mentioned algorithm is not available yet.

Although training has not been implemented yet, this is just to demonstrate the possibility of deploying the pose estimation model to a real-world application about autonomous driving. Future work directions include implementing the model within the simulation environment.

6. Conclusion

Although promising results are achieved, several directions for future research and improvements can be identified. Firstly, as larger datasets allows for better generalization, a potential improvement would be to integrate dataset with synthetic data. Secondly, the backbone could be also finetuned, instead of kept frozen and an enhanced data augmentation phase could be implemented. Furthermore, other padding modes could be explored to avoid padding also during inference as this increases inference time. The use of an iterative refinement module as in [1, 12] could also be considered while keeping the inference speed near real-time. Finally, the use of an attention module could also be explored, as in [1]. While this method can outperform [12], several aspects like training times and model size have to be addressed due to the increase of number of parameters.

Object	DenseFusion (per-pixel) (RGB-D)	Ours (baseline) (RGB)	Ours (baseline pipeline) (YOLO + RGB)	Ours (extension) (RGB-D)	Ours (extension pipeline) (YOLO + RGB-D)
ape (01)	79.5	0.0	0.0	51.1	31.7
bench vi. (02)	84.2	8.8	1.1	89.6	84.1
camera (04)	76.5	0.0	1.1	70.0	65.0
can (05)	86.6	2.2	1.7	81.0	80.5
cat (06)	88.8	1.1	0.0	79.7	82.0
driller (08)	77.7	3.9	0.5	90.5	86.5
duck (09)	76.3	0.0	0.0	52.1	47.9
eggbox (10)	99.8	13.3	0.5	100.0	100.0
glue (11)	99.4	18.0	6.0	100.0	100.0
hole p. (12)	79.0	1.1	0.5	59.7	61.3
iron (13)	92.1	2.9	0.5	95.4	88.4
lamp (14)	92.3	8.2	1.6	91.9	92.9
phone (15)	88.0	2.7	2.1	81.5	83.2
MEAN	86.2	4.8	1.9	80.0	77.0

Table 2. Comparison of 6D pose estimation methods proposed on the LineMOD dataset considering accuracy (%). In *Ours* columns, *pipeline* means models are evaluated on test set, with bounding boxes obtained from the object detection module, otherwise ground truth bounding boxes are considered.

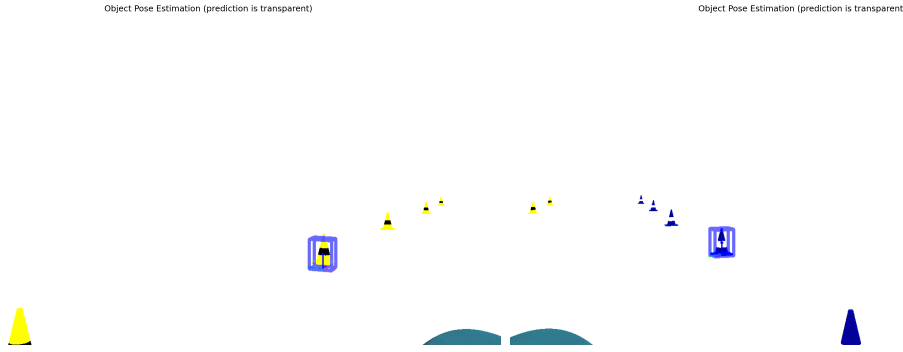


Figure 3. POV autonomous driving camera left and right. Images are obtained from a dummy dataset.

References

- [1] Weili Chen, Hui Zhang, Yiming Jiang, Yurong Chen, Bo Chen, and Changqing Huang. An Iterative Attention Fusion Network for 6D Object Pose Estimation. In *2023 China Automation Congress (CAC)*, pages 9300–9305, 2023. [2](#), [6](#)
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015. [1](#)
- [3] Stefan Hinterstoisser, Stefan Holzer, Cedric Cagniart, Slobodan Ilic, Kurt Konolige, Nassir Navab, and Vincent Lepetit. Multimodal templates for real-time detection of texture-less objects in heavily cluttered scenes. In *2011 International Conference on Computer Vision*, pages 858–865, 2011. [2](#)
- [4] Stefan Hinterstoisser, Vincent Lepetit, Slobodan Ilic, Stefan Holzer, Gary Bradski, Kurt Konolige, and Nassir Navab. Model Based Training, Detection and Pose Estimation of Texture-Less 3D Objects in Heavily Cluttered Scenes. In Kyoung Mu Lee, Yasuyuki Matsushita, James M. Rehg, and Zhanyi Hu, editors, *Computer Vision – ACCV 2012*, pages 548–562, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg. [5](#)
- [5] Nidhal Jegham, Chan Young Koh, Marwan Abdelatti, and Abdeltawab Hendawi. YOLO Evolution: A Comprehensive Benchmark and Architectural Review of YOLOv12, YOLO11, and Their Previous Versions, 2025. [1](#)
- [6] Wadim Kehl, Fabian Manhardt, Federico Tombari, Slobodan Ilic, and Nassir Navab. SSD-6D: Making RGB-based 3D detection and 6D pose estimation great again, 2017. [2](#)
- [7] Rahima Khanam and Muhammad Hussain. Yolov11: An Overview of the Key Architectural Enhancements, 2024. [1](#)
- [8] Giorgia Marullo, Leonardo Tanzi, Pietro Piazzolla, and Enrico Vezzetti. 6D object position estimation from 2D images: a literature review. *Multimedia Tools and Applications*, 2023. [1](#), [2](#)
- [9] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation, 2017. [1](#)

- [10] Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. Point-Net++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space, 2017. 1, 3
- [11] Ultralytics. Ultralytics YOLOv11 Documentation. <https://docs.ultralytics.com>, 2024. Accessed: 2025-07-06. 1, 2
- [12] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martín-Martín, Cewu Lu, Li Fei-Fei, and Silvio Savarese. DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion, 2019. 1, 2, 3, 6
- [13] Yu Xiang, Tanner Schmidt, Venkatraman Narayanan, and Dieter Fox. PoseCNN: A Convolutional Neural Network for 6D Object Pose Estimation in Cluttered Scenes, 2018. 2, 5
- [14] Chen Yuanwei, Mohd Hairi Mohd Zaman, and Mohd Faisal Ibrahim. A Review on Six Degrees of Freedom (6D) Pose Estimation for Robotic Applications. *IEEE Access*, 12:161002–161017, 2024. 1, 5